

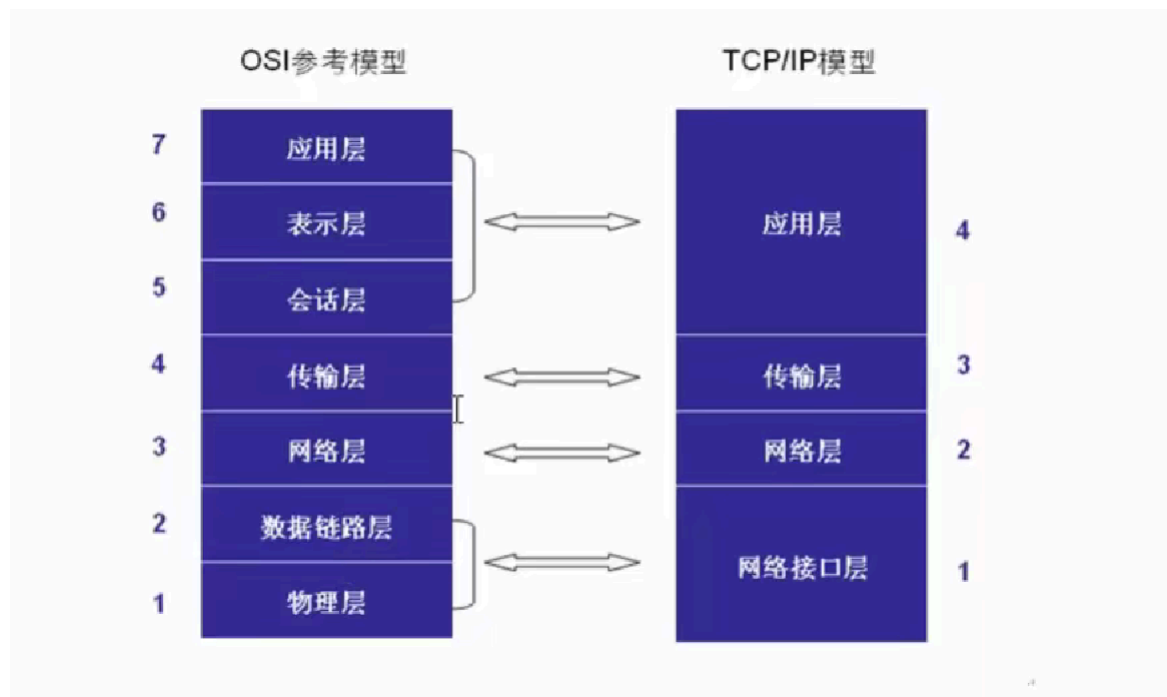
网络基础

该部分都是理论知识，可以去看计网书籍

1. 协议的概念

一组规则，两端都遵循这个协议来进行传输

2. 七层模型和四层模型



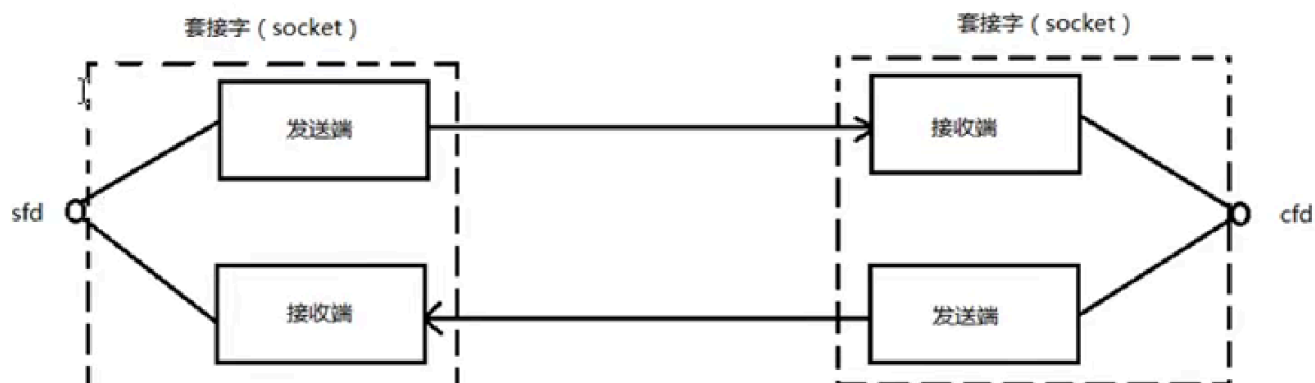
Socket编程

1. 套接字

在通信过程中，**套接字一定是成对出现的**

一个文件描述符指向一个套接字(该套接字内部由内核借助两个缓冲区实现读写)

Linux套接字实现原理：



2. 预备知识

2.1 网络字节序

内存中的多字节数据相对于内存地址有大端和小端之分

小端法：(PC本地存储) 高位存高地址，低位存低地址

大端法：(网络存储) 高位存低地址，低位存高地址

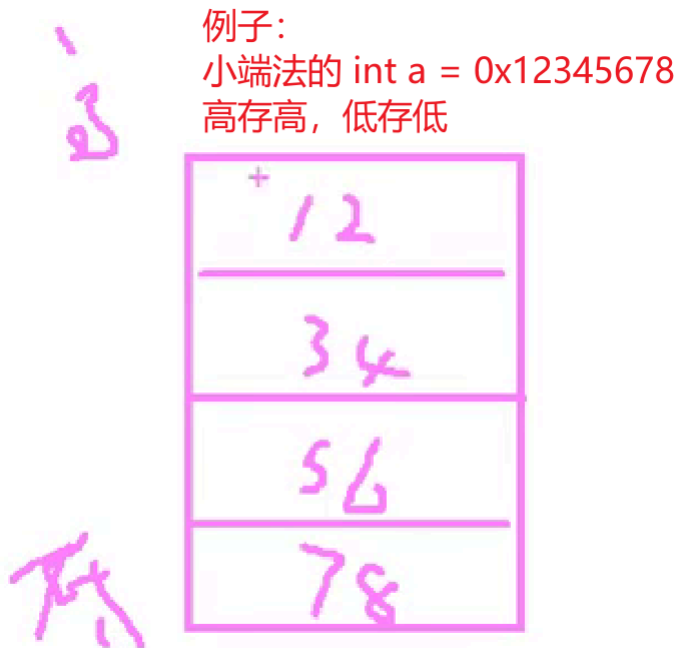
网络的数据流采用大端字节序，而本地的数据流采用小端字节序，因此要通过函数来转换：

htonl --> 本地 --> 网络 (IP)

htons --> 本地 --> 网络 (port端口)

ntohl --> 网络 --> 本地 (IP)

ntohs --> 网络 --> 本地 (port)



2.2 IP地址转换函数

IP地址转换函数:

```
int inet_pton(int af, const char *src, void *dst);           本地字节序 (string IP) ---> 网络字节序

af: AF_INET、AF_INET6
src: 传入, IP地址 (点分十进制)
dst: 传出, 转换后的 网络字节序的 IP地址。
返回值:
    成功: 1
    异常: 0, 说明src指向的不是一个有效的ip地址。
    失败: -1

const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);   网络字节序 ---> 本地字节序 (string IP)

af: AF_INET、AF_INET6
src: 网络字节序IP地址
dst: 本地字节序 (string IP)
size: dst 的大小。
```

2.3 sockaddr数据结构

```
#include <arpa/inet.h>
```

```
struct sockaddr_in {
```

```
    sa_family_t    sin_family;

    in_port_t      sin_port;

    struct in_addr  sin_addr;

};
```

1. sin_family: 表示你要使用的地址结构类型, AF_INET是IPv4, AF_INET6是IPv6;
2. sin_port: 网络字节序的端口号, 因此要使用htons转换一下;
3. struct in_addr sin_addr: 一个结构体, 里面有一个s_addr, 要传入网络字节序的ip地址, 因此要使用inet_pton函数; 也可以使用第二种方法, 宏

sockaddr地址结构:

```
struct sockaddr_in addr;

addr.sin_family = AF_INET/AF_INET6

addr.sin_port = htons(9527);

int dst;

inet_pton(AF_INET, "192.157.22.45", (void *)&dst);
addr.sin_addr.s_addr = dst;

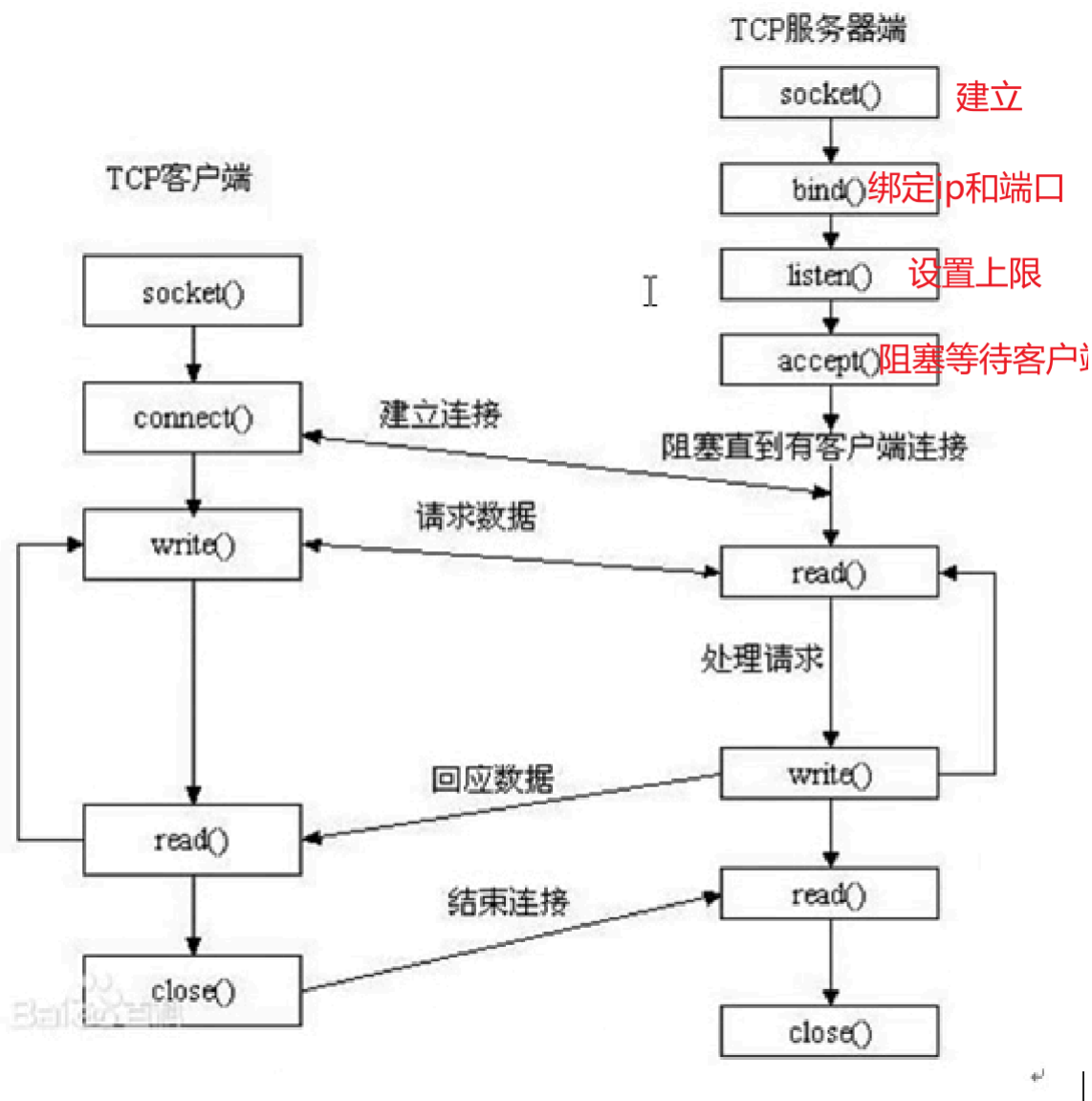
【*】addr.sin_addr.s_addr = htonl(INADDR_ANY);           取出系统中有效的任意IP地址。二进制类型。

bind(fd, (struct sockaddr *)&addr, size);
```

3. 网络套接字函数

3.1 socket模型创建流程分析

当服务器端调用socket函数时会产生一个套接字，而accept函数会阻塞监听客户端连接，当有客户端过来连接的时候 该函数会返回一个新的套接字去和客户端连接，因此一个socket建立会有两个套接字(socket函数产生的起监听作用，accept函数返回的，客户端的)



3.2 socket 函数

头文件:

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

语法:

```
int socket(int domain, int type, int protocol); 创建一个套接字
```

domain: AF_INET或AF_INET6

type: 数据传输协议 SOCK_STREAM或SOCK_DGRAM

protocol: 默认传0, 根据type来选择, SOCK_STREAM的代表协议是TCP, SOCK_DGRAM的是UDP

返回值:

成功: 新套接字所对应的文件描述符; 失败: -1 error

3.3 bind 函数

语法:

`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);` 给socket绑定地址结构

sockfd: socket函数的返回值

addr: 传入参数 (struct sockaddr *)&addr

```
struct sockaddr_in addr;  
addr.sin_family = AF_INET;  
addr.sin_port = htons(8888);  
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

addrlen: sizeof(addr) 地址结构的大小

返回值:

成功是0, 失败-1 error

3.4 listen 函数

语法:

`int listen(int sockfd, int backlog);` 设置同时与服务器建立连接的上限数(同时进行3次握手的客户端数量);

sockfd: socket的返回值

backlog: 上限数值

返回值:

成功是0, 失败-1 error

3.5 accept 函数

语法:

`int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);` 阻塞等待客户端建立连接, 成功的话 返回一个与客户端成功连接的socket文件描述符。

sockfd: socket的返回值

addr: 传出参数, 成功与服务器建立连接的那个**客户端**的地址结构(ip+端口)

addrlen: &clit_addr_len 传入传出参数, 入: addr的大小 出: 客户端addr的实际大小

```
socklen_t  clit_addr_len = sizeof(addr);  &clit_addr_len
```

返回值:

成功: 能与服务器进行数据通信的 socket 对应的文件描述符

失败: -1 error

3.6 connect 函数

语法:

`int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);` 使用客户端现有的 socket 与服务器进行连接。

sockfd: socket的返回值

addr: 服务器的地址结构

addrlen: 服务器地址结构的长度

返回值:

成功: 0

失败: -1 error

3.7 通信案例的实现

```
#include <stdio.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <pthread.h>

//端口号
#define SERV_PORT 9527

int main(){
```

```
char buf[BUFSIZ];
int ret,i;

int lfd = 0, cfd = 0;

struct sockaddr_in serv_addr, clit_addr;
socklen_t clit_addr_len;

serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(SERV_PORT);
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);

//1. 创建socket
lfd = socket(AF_INET,SOCK_STREAM,0);
if(lfd == -1)
{
    perror("socket error");
    exit(1);
}
//2. 绑定ip和端口
bind(lfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr));
//3. 设置上限
listen(lfd,128);
//4. 阻塞等待客户端连接
clit_addr_len = sizeof(clit_addr);
cfd = accept(lfd,(struct sockaddr *)&clit_addr,&clit_addr_len);
if(cfd==-1)
{
    perror("accept error");
    exit(1);
}
//5. 读写数据
while(1){
    ret = read(cfd,buf,sizeof(buf));
    write(STDOUT_FILENO,buf,ret);

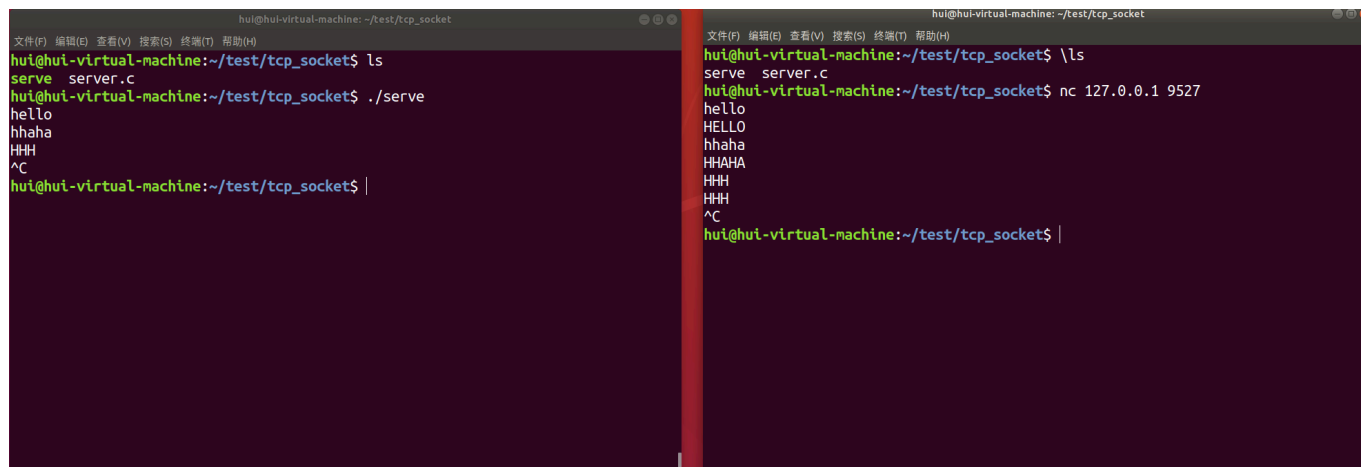
    for(i=0;i<ret;i++)
    {
        buf[i] = toupper(buf[i]);
    }

    write(cfd,buf,ret);
}

//关闭
close(lfd);
close(cfd);

return 0;
}
```

可以通过 nc+ip地址+端口号进行连接测试，可以看到能够正常的小写转大写



The image shows two terminal windows side-by-side. The left window is titled 'hui@hui-virtual-machine: ~/test/tcp_socket' and shows the execution of a server program. The user runs 'ls', then 'serve server.c', and finally './serve'. The program outputs 'hello', 'hhaha', 'HHH', and '^C' before returning to the prompt. The right window is titled 'hui@hui-virtual-machine: ~/test/tcp_socket' and shows the execution of a client program. The user runs '\ls', then 'serve server.c', and finally 'nc 127.0.0.1 9527'. The program outputs 'hello', 'HELLO', 'hhaha', 'HHAHA', 'HHH', 'HHH', and '^C' before returning to the prompt.

```
#include <stdio.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <pthread.h>

#define SERV_PORT 9527

int main()
{
    int cfd = 0;
    char buf[BUFSIZ];
    int conter = 11;

    struct sockaddr_in serv_addr;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(SERV_PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr.s_addr);

    //创建socket
    cfd = socket(AF_INET, SOCK_STREAM, 0);
    if(cfd == -1)
    {
        perror("socket error");
        exit(1);
    }

    // connect连接
    int ret = connect(cfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    if(ret == -1)
    {
        perror("connect error");
        exit(1);
    }
}
```

```
hui@hui-virtual-machine: ~/test/tcp_socket  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
hui@hui-virtual-machine:~/test/tcp_socket$ ls  
client client.c serve server.c  
hui@hui-virtual-machine:~/test/tcp_socket$ ./serve  
client ip:127.0.0.1 port:34440  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello  
hello
```

```
hui@hui-virtual-machine: ~/test/tcp_socket  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
hui@hui-virtual-machine:~/test/tcp_socket$ ls  
client client.c serve server.c  
hui@hui-virtual-machine:~/test/tcp_socket$ ./client  
HELLO  
HELLO  
HELLO  
HELLO  
HELLO  
HELLO  
HELLO  
^ [HELLO  
HELLO  
HELLO  
hui@hui-virtual-machine:~/test/tcp_socket$
```

多进程并发服务器

核心思路：让accept循环阻塞等待客户端，每当有客户端连接时就fork子进程，让子进程去和客户端进行通信，父进程用于监听并使用信号捕捉回收子进程；(子进程关闭用于监听的套接字lfd，父进程关闭用于通信的cfd)

9 / 55

```
//捕捉信号的回调函数
void catch_child(int sigsum)
{
    while((waitpid(0,NULL,WNOHANG))>0);
    return ;
}
int main()
{
    int lfd,cfd;
    pid_t pid;
    char buf[BUFSIZ];
    int ret, i;

    //地址结构体
    struct sockaddr_in serve_addr, client_addr;
    socklen_t client_addr_len;

    bzero(&serve_addr,sizeof(serve_addr)); //将地址结构清零
    serve_addr.sin_family = AF_INET;
    serve_addr.sin_port = htons(SERVE_PORT);
    serve_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    //1. 创建socket
    lfd = socket(AF_INET,SOCK_STREAM,0);
    if(lfd == -1)
    {
        perror("socket error");
        exit(1);
    }
    //2. 绑定ip和端口
    bind(lfd,(struct sockaddr *)&serve_addr,sizeof(serve_addr));
    //3. 设置上限
    listen(lfd,128);
    //4. 子进程用于通信, 父进程监听并回收子进程
    client_addr_len = sizeof(client_addr);
    while(1)
    {
        //这里的accept会阻塞等待
        cfd = accept(lfd,(struct sockaddr
*&client_addr,&client_addr_len);

        pid = fork();
        if(pid<0)
        {
            perror("fork error");
            exit(1);
        }
        else if(pid==0) //子进程
        {
            close(lfd); //关闭用于监听的套接字
            //进程的地址空间是独立的, 这是关闭的是进程中的lfd, 父进程的还是正常
```

监听中

```

        for(;;){
            ret = read(cfd,buf,sizeof(buf)); //读数据

            //如果ret=0就说明客户端那边关闭了连接，我们就关闭cfd 退
出进程

            if(ret == 0)
            {
                close(cfd);
                exit(1);
            }

            for(i=0;i<ret;i++)
                buf[i] = toupper(buf[i]);    //改数据

            write(cfd,buf,ret);    //写回数据
            write(STDOUT_FILENO,buf,ret);
        }
    }
    else    //父进程
    {
        //信号回收子进程
        struct sigaction act;
        act.sa_handler = catch_child;
        sigemptyset(&act.sa_mask);
        act.sa_flags = 0;

        sigaction(SIGCHLD,&act,NULL);

        close(cfd);    //关闭用于通信的套接字
        continue;
    }
}

return 0;
}

```

多线程并发服务器

```

1.#include <stdio.h>
2.#include <string.h>
3.#include <arpa/inet.h>
4.#include <pthread.h>
5.#include <ctype.h>
6.#include <unistd.h>
7.#include <fcntl.h>
8.
9.#include "wrap.h"
10.
11.#define MAXLINE 8192
12.#define SERV_PORT 8000

```

```

13.
14. struct s_info {                                //定义一个结构体，将地址结构跟cfd捆绑
15.     struct sockaddr_in cliaddr;
16.     int connfd;
17. };
18.
19. void *do_work(void *arg)
20. {
21.     int n,i;
22.     struct s_info *ts = (struct s_info*)arg;
23.     char buf[MAXLINE];
24.     char str[INET_ADDRSTRLEN];                // #define INET_ADDRSTRLEN 16 可用"+d"查看
25.
26.     while (1) {
27.         n = Read(ts->connfd, buf, MAXLINE);    //读客户端
28.         if (n == 0) {
29.             printf("the client %d closed...\n", ts->connfd);
30.             break;                             //跳出循环,关闭
cfd
31.         }
32.         printf("received from %s at PORT %d\n",
33.             inet_ntop(AF_INET, &(*ts).cliaddr.sin_addr, str, sizeof(str)),
34.             ntohs((*ts).cliaddr.sin_port));    //打印客户端信息
(IP/PORT)
35.
36.         for (i = 0; i < n; i++)
37.             buf[i] = toupper(buf[i]);         //小写-->大写
38.
39.         Write(STDOUT_FILENO, buf, n);         //写出至屏幕
40.         Write(ts->connfd, buf, n);            //回写给客户端
41.     }
42.     Close(ts->connfd);
43.
44.     return (void *)0;
45. }
46.
47. int main(void)
48. {
49.     struct sockaddr_in servaddr, cliaddr;
50.     socklen_t cliaddr_len;
51.     int listenfd, connfd;
52.     pthread_t tid;
53.
54.     struct s_info ts[256];                    //创建结构体数组.
55.     int i = 0;
56.
57.     listenfd = Socket(AF_INET, SOCK_STREAM, 0); //创建一个
socket, 得到lfd
58.
59.     bzero(&servaddr, sizeof(servaddr));       //地址结构清
零
60.     servaddr.sin_family = AF_INET;
61.     servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
//指定本地任意IP

```

```

62.     servaddr.sin_port = htons(SERV_PORT);
    //指定端口号
63.
64.     Bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    //绑定
65.
66.     Listen(listenfd, 128);
    //设置同一时刻链接服务器上限数
67.
68.     printf("Accepting client connect ...\n");
69.
70.     while (1) {
71.         cliaddr_len = sizeof(cliaddr);
72.         connfd = Accept(listenfd, (struct sockaddr *)&cliaddr, &cliaddr_len);
    //阻塞监听客户端链接请求
73.         ts[i].cliaddr = cliaddr;
74.         ts[i].connfd = connfd;
75.
76.         pthread_create(&tid, NULL, do_work, (void *)&ts[i]);
77.         pthread_detach(tid);
    //子线程分离,防止僵线程产生.
78.         i++;
79.     }
80.
81.     return 0;
82.}

```

端口复用

在socket()和bind()之间写入

端口复用:

```

int opt = 1; // 设置端口复用。
setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, (void *)&opt, sizeof(opt));

```

I/O 多路复用

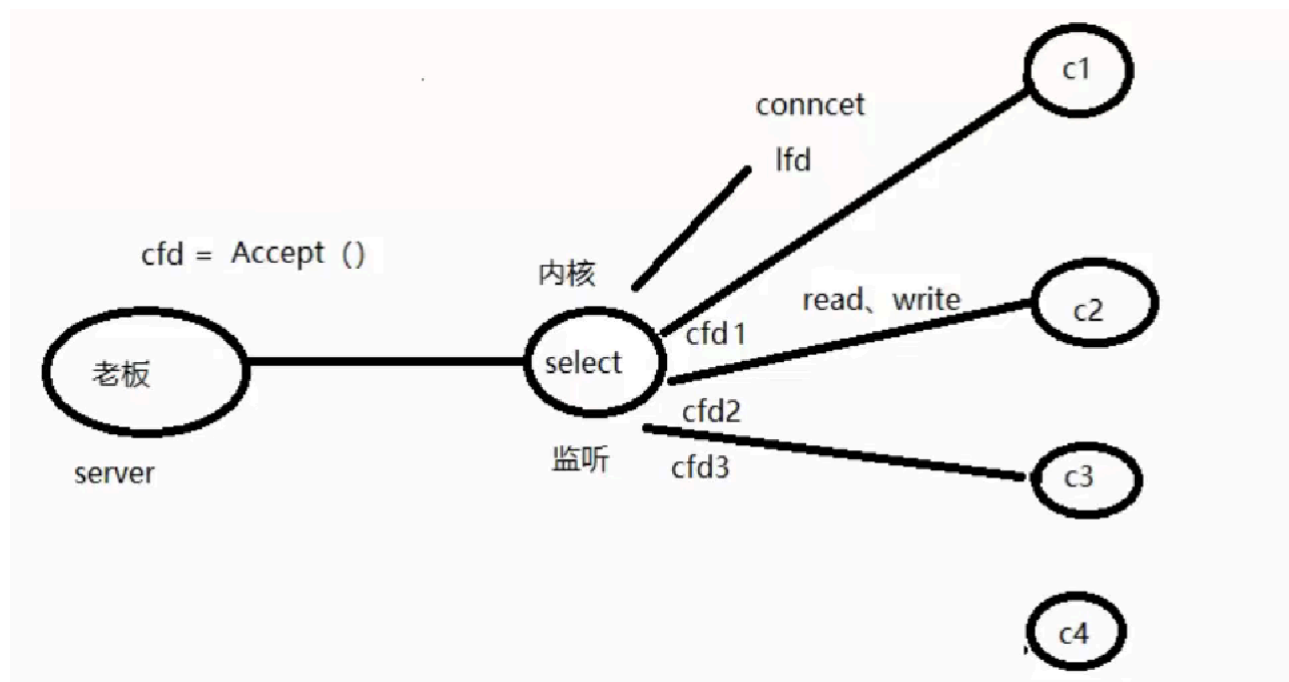
多路 IO 转接服务器也叫做多任务 IO 服务器。该类服务器实现的主旨思想是，不再由应用程序自己监视客户端连接，取而代之由内核替应用程序监视文件。

主要使用的方法有三种

1. select
2. poll
3. epoll

select

让内核去监听客户端连接(lfd)，当有客户端进行连接时 它会让server去调用accept(当有连接 时才去立即调用，而不是一直阻塞等待)得到一个用于通信的cfd，最后让内核监管着lfd和所有cfd



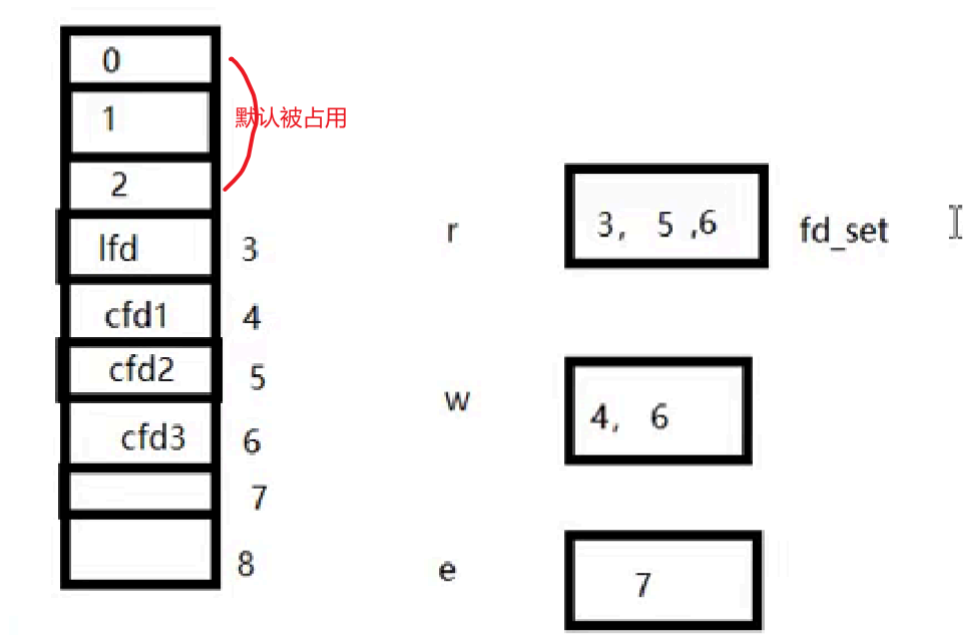
函数解析

底层原理：

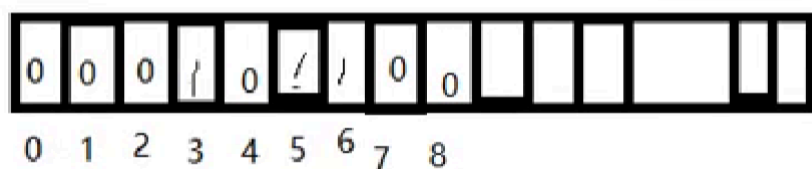
文件描述符表：前三个默认被系统占用

fd_set集合：传入的是文件描述符，传出所有监听集合(读、写、异常)中满足对应事件的总数

fd_set集合的本质：位图(二进制位存放文件描述符的状态)，默认都为0，若发生变化就置1



位图中 文件描述符3 5 6的状态发生了变化，所以置1



语法：

```
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

- nfds：监听的所有文件描述符中，最大文件描述符+1；
- readfds：读 文件描述符监听集合。传入传出参数
- writefds：写 文件描述符监听集合。传入传出参数，通常传NULL
- exceptfds：异常 文件描述符监听集合。传入传出参数，通常传NULL
- timeout：大于0表示设置监听时长，NULL表示阻塞监听，0表示非阻塞监听 while轮询

返回值：

- 大于0：所有监听集合(读、写、异常)中满足对应事件的总数
- 0：没有满足监听条件的文件描述符
- -1：error

监听集合对应函数：

1. void FD_ZERO(fd_set *set); --清空一个文件描述符集合

```
fd_set rset;    FD_ZERO(&rset);    //将rset集合清空
```

2. void FD_SET(int fd, fd_set *set); --将待监听的文件描述符添加到监听集合中

```
FD_SET(3,&rset); FD_SET(5,&rset);    //将文件描述符3和5加到rset集合中
```

3. `void FD_CLR(int fd, fd_set *set);` --将一个文件描述符从监听集合中移除

4. `int FD_ISSET(int fd, fd_set *set);` --判断一个文件描述符是否在该集合中

实现思路

如果在服务器基于 select 实现并发，其处理流程如下：

```
int maxfd = 0;

lfd = socket() ;           创建套接字

maxfd = lfd;

bind();                   绑定地址结构

listen();                 设置监听上限

fd_set rset,  allset;      创建r监听集合

FD_ZERO(&allset);         将r监听集合清空

FD_SET(lfd, &allset);     将 lfd 添加至读集合中。

while (1) {

    rset = allset;         保存监听集合

    ret = select(lfd+1, &rset, NULL, NULL, NULL);    监听文件描述符集合对
应事件。

    if (ret > 0) {         有监听的描述符满足对应事件

        if (FD_ISSET(lfd, &rset)) {                // 1 在。 0不在。

            cfd = accept ();                        建立连接，返回用于通信的文件描述符

            maxfd = cfd;

            FD_SET(cfd, &allset);                   添加到监听通信描述符集合中。

        }

        for (i = lfd+1; i <= 最大文件描述符; i++) {

            FD_ISSET(i, &rset)                      有read、write事件

            read ()
```

```

        小 -- 大

        write();
    }
}
}

```

代码实现

```

#include <stdio.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/time.h>
#include <sys/select.h>

#define SERVE_PORT 9527

int main()
{
    int lfd, cfd, ret, len, j, i;
    char buf[BUFSIZ];

    //地址结构体
    struct sockaddr_in serve_addr, client_addr;
    socklen_t client_addr_len;

    bzero(&serve_addr, sizeof(serve_addr));    //结构体清空

    serve_addr.sin_family = AF_INET;
    serve_addr.sin_port = htons(SERVE_PORT);
    serve_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    //创建socket
    lfd = socket(AF_INET, SOCK_STREAM, 0);
    int maxfd = lfd;    //最大的文件描述符
    if(lfd == -1)
    {
        perror("socket error");
        exit(1);
    }
    //绑定ip和端口
    bind(lfd, (struct sockaddr *)&serve_addr, sizeof(serve_addr));
    //设置上限
    listen(lfd, 128);

```

```

fd_set rset, allset;           //设置监听的集合
FD_ZERO(&allset);              //清空集合
FD_SET(lfd,&allset);           //将lfd加入到监听集合

while(1)
{
    rset = allset;
    ret = select(maxfd+1,&rset,NULL,NULL,NULL);

    if(ret<0)
    {
        perror("select error");
        exit(1);
    }
    if(FD_ISSET(lfd,&rset))      //如果lfd在传出的rset中，表示有客户端要进
行连接
    {
        client_addr_len = sizeof(client_addr);
        cfd = accept(lfd,(struct sockaddr
*)&client_addr,&client_addr_len);

        FD_SET(cfd,&allset);    //将cfd加入到监听集合

        if(maxfd<cfd)
            maxfd = cfd;        //更新最大的文件描述符

        //说明select只返回了lfd这一个，后续指令无需执行，continue跳出本次，
继续while

        if(ret==1)
            continue;
    }

    //如果ret!=1 说明还监听到了其他描述符的read事件
    for(i=lfd+1; i<=maxfd; i++)
    {
        if(FD_ISSET(i,&rset)){    //找到满足读事件的那个描述符
            len = read(i,buf,sizeof(buf));
            if(len==0)            //检测到客户端关闭了连接
            {
                close(i);        //关闭该描述符
                FD_CLR(i,&allset); //将该描述符从监听集合中
移除

            }

            //如果len不为0就表示有数据，循环更改数据
            for(j=0; j<len; j++)
                buf[j] = toupper(buf[j]);

            //写回更改后的数据
            write(i,buf,len);
            write(STDOUT_FILENO,buf,len);
        }
    }
}

```

```

        }

    }

    close(lfd);
    return 0;
}

```

关于rset和allset的解析：因为select函数返回出来的是满足条件的，如果我们将新的cfd也加入到rset中，那么等到下次循环时，如果刚刚加入的cfd没有读写数据发生时，就会被踢出rset，此时我们需要用allset来存储新进来的cfd，每次进while循环后就赋值给rset；

poll

相对select没有太大改进，属于半成品，比较鸡肋 了解即可。

函数解析

语法：

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

poll:

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
    fds: 监听的文件描述符【数组】
        struct pollfd {
            int fd:        待监听的文件描述符
            short events:   待监听的文件描述符对应的监听事件
                           取值: POLLIN、POLLOUT、POLLERR
            short revents:  传入时，给0。如果满足对应事件的话，返回非0 --> POLLIN、POLLOUT、POLLERR
        }
    nfds: 监听数组的，实际有效监听个数。
    timeout: > 0: 超时长。单位：毫秒。
            -1: 阻塞等待
            0: 不阻塞
    返回值: 返回满足对应监听事件的文件描述符 总个数。|
```

代码实现

这个东西用得少，基本都用epoll，从讲义上挂个代码过来，看看视频里思路就完事儿

```

1./* server.c */
2.#include <stdio.h>
3.#include <stdlib.h>
4.#include <string.h>
5.#include <netinet/in.h>
6.#include <arpa/inet.h>
7.#include <poll.h>

```

```

8.#include <errno.h>
9.#include "wrap.h"
10.
11.#define MAXLINE 80
12.#define SERV_PORT 6666
13.#define OPEN_MAX 1024
14.
15.int main(int argc, char *argv[])
16.{
17.    int i, j, maxi, listenfd, connfd, sockfd;
18.    int nready;
19.    ssize_t n;
20.    char buf[MAXLINE], str[INET_ADDRSTRLEN];
21.    socklen_t clilen;
22.    struct pollfd client[OPEN_MAX];
23.    struct sockaddr_in cliaddr, servaddr;
24.
25.    listenfd = Socket(AF_INET, SOCK_STREAM, 0);
26.
27.    bzero(&servaddr, sizeof(servaddr));
28.    servaddr.sin_family = AF_INET;
29.    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
30.    servaddr.sin_port = htons(SERV_PORT);
31.
32.    Bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
33.
34.    Listen(listenfd, 20);
35.
36.    client[0].fd = listenfd;
37.    client[0].events = POLLRDNORM;          /* listenfd监听普通读事件 */
38.
39.    for (i = 1; i < OPEN_MAX; i++)
40.        client[i].fd = -1;                /* 用-1初始化client[]里剩下
元素 */
41.    maxi = 0;                             /* client[]数组有效元素中最
大元素下标 */
42.
43.    for ( ; ; ) {
44.        nready = poll(client, maxi+1, -1); /* 阻塞 */
45.        if (client[0].revents & POLLRDNORM) { /* 有客户端链接请求 */
46.            clilen = sizeof(cliaddr);
47.            connfd = Accept(listenfd, (struct sockaddr *)&cliaddr, &clilen);
48.            printf("received from %s at PORT %d\n",
49.                inet_ntop(AF_INET, &cliaddr.sin_addr, str, sizeof(str)),
50.                ntohs(cliaddr.sin_port));
51.            for (i = 1; i < OPEN_MAX; i++) {
52.                if (client[i].fd < 0) {
53.                    client[i].fd = connfd; /* 找到client[]中空闲的位置, 存放
accept返回的connfd */
54.                    break;
55.                }
56.            }
57.
58.            if (i == OPEN_MAX)

```

```

59.         perr_exit("too many clients");
60.
61.         client[i].events = POLLRDNORM;      /* 设置刚刚返回的connfd, 监控读事
件 */
62.         if (i > maxi)
63.             maxi = i;                      /* 更新client[]中最大元素下标 */
64.         if (--nready <= 0)
65.             continue;                     /* 没有更多就绪事件时,继续回到
poll阻塞 */
66.     }
67.     for (i = 1; i <= maxi; i++) {          /* 检测client[] */
68.         if ((sockfd = client[i].fd) < 0)
69.             continue;
70.         if (client[i].revents & (POLLRDNORM | POLLERR)) {
71.             if ((n = Read(sockfd, buf, MAXLINE)) < 0) {
72.                 if (errno == ECONNRESET) { /* 当收到 RST标志时 */
73.                     /* connection reset by client */
74.                     printf("client[%d] aborted connection\n", i);
75.                     Close(sockfd);
76.                     client[i].fd = -1;
77.                 } else {
78.                     perr_exit("read error");
79.                 }
80.             } else if (n == 0) {
81.                 /* connection closed by client */
82.                 printf("client[%d] closed connection\n", i);
83.                 Close(sockfd);
84.                 client[i].fd = -1;
85.             } else {
86.                 for (j = 0; j < n; j++)
87.                     buf[j] = toupper(buf[j]);
88.                 Writen(sockfd, buf, n);
89.             }
90.             if (--nready <= 0)
91.                 break;                     /* no more readable descriptors */
92.         }
93.     }
94. }
95. return 0;
96. }

```

epoll★

突破文件描述符限制

cat /proc/sys/fs/file-max; --查看计算机最大打开文件数

ulimit -a; --查看计算机每个进程所能使用的文件描述符, 默认是1024

cat /proc/sys/fs/file-max --查看最大文件描述符上限

修改上限:

修改：

打开 `sudo vi /etc/security/limits.conf`，写入：

```
* soft nfile 65536          --> 设置默认值， 可以直接借助命令修改。 【注销用户，使其生效】  
  
* hard nfile 100000         --> 命令修改上限。
```

函数解析

其本质是红黑树和链表

1. `int epoll_create(int size);`

size：创建的红黑树的监听节点数量(仅供内核参考)

返回值：成功--指向新创建的红黑树的**根节点**的fd；失败 -1 error

2. `int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);`

epfd：epoll_create函数的返回值 epfd

op：对该监听红黑树所要做的操作

EPOLL_CTL_ADD 添加fd到监听红黑树

EPOLL_CTL_MOD 修改fd在监听红黑树上的监听事件

EPOLL_CTL_DEL 将一个fd从监听红黑树上摘下(取消监听)

fd：待监听的fd

event：本质是 struct epoll_event 结构体地址

events：EPOLLIN、EPOLLOUT、EPOLLERR

data：联合体

int fd：对应监听事件的fd

void *ptr：暂时不写

uint32_t u32

uint32_t u64

返回值：成功0，失败-1 error

3. `int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);`

epfd：epoll_create函数的返回值 epfd

events：传出参数，一个数组[]，用于存放满足监听条件的那些fd结构体

maxevents: 数组元素的总个数 struct epoll_event events[1024]

timeout:

-1: 阻塞

0: 不阻塞

0: 超时时间(毫秒)

返回值:

0: 满足监听的总个数, 可以用作循环上限

0: 没有fd满足监听的事件

-1: 失败, error

核心思路

1. socket()、bind()、listen()
2. epoll_create创建红黑树, 它的返回值就是树的根节点
3. epoll_ctl将listenfd添加到树上
4. 循环epoll_wait进行监听, 它的返回值是满足监听的总个数, 所以以它的返回值为遍历上限去判断事件
5. 如果它返回的数组中data.fd等于lfd, 那么就accept去连接客户端 并将新的cfd加入树中
6. 如果不是lfd,就说明有读事件发生, 就去判断读到的返回值, <0是出错 ==0是客户端关闭(这两个都要去将该cfd从树中移除 并close), >0就处理数据然后写回

代码实现

```
#include <stdio.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/epoll.h>
#include <fcntl.h>

#define OPEN_MAX 5000
#define SERVE_PORT 9527

int main()
{
    // 所需要的变量
    int lfd, cfd, efd, ret, wait_ret, i, sockfd, len;
    char buf[1024];
    // 地址结构体
```

```
struct sockaddr_in serve_addr, client_addr;
socklen_t client_addr_len;
serve_addr.sin_family = AF_INET;           // IPV4
serve_addr.sin_port = htons(SERVE_PORT);   // 绑定端口
serve_addr.sin_addr.s_addr = htonl(INADDR_ANY); // 绑定ip(ANY系统自动分配)

// 创建socket
lfd = socket(AF_INET, SOCK_STREAM, 0);
if (lfd < 0)
{
    perror("socket error");
    exit(1);
}
// bind绑定
bind(lfd, (struct sockaddr *)&serve_addr, sizeof(serve_addr));
// 设置上限
listen(lfd, 128);

// 创建红黑树
efd = epoll_create(1); // efd就是树的根节点

// 将lfd挂在树上
// epoll结构体                                ep是epoll_wait所需的数组(存放满足事件的fd)
struct epoll_event tep, ep[128]; // tep是epoll_ctl的参数(传监听的事件)
tep.events = EPOLLIN;
tep.data.fd = lfd;

ret = epoll_ctl(efd, EPOLL_CTL_ADD, lfd, &tep);
if (ret < 0)
{
    perror("epoll_ctl error");
    exit(1);
}

// 循环去epoll_wait进行监听
while (1)
{
    wait_ret = epoll_wait(efd, ep, 128, -1); // wait_ret就是实际满足事件的总个数

    // 以wait_ret为上限去遍历事件
    for (i = 0; i < wait_ret; i++)
    {
        // sockfd用于接收满足事件的fd
        sockfd = ep[i].data.fd;

        // 如果等于lfd,那就说明有客户端要来连接,就去accept
        if (sockfd == lfd)
        {
            client_addr_len = sizeof(client_addr);
            cfd = accept(lfd, (struct sockaddr *)&client_addr,
            &client_addr_len);

            // 将cfd设置为非阻塞
            int flag = fcntl(cfd, F_GETFL);
```

```

        flag |= O_NONBLOCK;
        fcntl(cfd, F_SETFL, flag);

        // 把新的cfd加入树中
        tep.events = EPOLLIN | EPOLLET;
        tep.data.fd = cfd;
        ret = epoll_ctl(efd, EPOLL_CTL_ADD, cfd, &tep);
        if (ret < 0)
        {
            perror("epoll_ctl cfd error");
            exit(1);
        }
    }
    // 如果不是lfd,那就说明有读事件发生(读数据)
    else
    {
        len = read(sockfd, buf, sizeof(buf));
        if (len == 0) // 说明对方关闭连接(从树上摘下 & close)
        {
            epoll_ctl(efd, EPOLL_CTL_DEL, sockfd, NULL);
            close(sockfd);
        }
        else if (len == -1)
        {
            perror("read error");
            exit(1);
        }
        else // 读写数据
        {
            for (i = 0; i < len; i++)
                buf[i] = toupper(buf[i]);

            write(sockfd, buf, len);
            write(STDIN_FILENO, buf, len);
        }
    }
}

return 0;
}

```

事件模型

epoll事件有两种模型：

1. ET边缘触发(event.events = EPOLLIN | EPOLLET)：只有数据到来才触发，不管缓存区中是否还有数据
- 使用边缘触发模式时，当被监控的 Socket 描述符上有可读事件发生时，**服务器端只会从 epoll_wait 中苏醒一次**，即使进程没有调用 read 函数从内核读取数据，也依然只苏醒一次，因此我们程序要保证一次性将内核缓冲区的数据读取完；

2. LT水平触发(默认 event.events = EPOLLIN): 只要有数据都会触发

- 使用水平触发模式时, 当被监控的 Socket 上有可读事件发生时, **服务器端不断地从 `epoll_wait` 中苏醒, 直到内核缓冲区数据被 `read` 函数读完才结束**, 目的是告诉我们有数据需要读取;

举个例子, 你的快递被放到了一个快递箱里, 如果快递箱只会通过短信通知你一次, 即使你一直没有去取, 它也不会再发送第二条短信提醒你, 这个方式就是边缘触发; 如果快递箱发现你的快递没有被取出, 它就会不停地发短信通知你, 直到你取出了快递, 它才消停, 这个就是水平触发的方式。

这就是两者的区别, 水平触发的意思是只要满足事件的条件, 比如内核中有数据需要读, 就一直不断地把这个事件传递给用户; 而边缘触发的意思是只有第一次满足条件的时候才触发, 之后就不会再传递同样的事件了。

一般来说, **边缘触发的效率比水平触发的效率要高**, 因为边缘触发可以减少 `epoll_wait` 的系统调用次数

ET的非阻塞模式

如果使用 `epoll` 的边缘模式进行读事件的检测, 有新数据达到只会通知一次, 那么必须要保证得到通知后将数据全部从读缓冲区中读出。那么, 应该如何读这些数据呢?

`epoll` 在边缘模式下, 必须要将套接字设置为非阻塞模式

```
// 设置完成之后, 读写都变成了非阻塞模式
//accept后, epoll_ctl挂载前
//将cfd设置为非阻塞
int flag = fcntl(cfd, F_GETFL);
flag |= O_NONBLOCK;
fcntl(cfd, F_SETFL, flag);
```

chatgpt: 当一个文件描述符 (如`cfd`) 被设置为非阻塞模式后, 这个描述符在所有的操作中都将表现为非阻塞的。

在你的代码中, 将`cfd`设置为非阻塞模式后, 只有`cfd`对应的套接字发生就绪事件 (如有数据可读) 时, `epoll_wait`才会立即返回, 并返回就绪的事件数量。如果没有就绪事件, `epoll_wait`将立即返回0, 而不会等待阻塞。

所以, 将`cfd`设置为非阻塞模式会影响到`epoll_wait`的行为, 使其变成非阻塞的, 即使没有就绪的事件也不会阻塞等待。这样可以提高效率, 避免不必要的阻塞等待, 提高程序的响应性。

epoll反应堆模型

核心: `epoll ET模式+非阻塞+void *ptr`

event: 本质是 `struct epoll_event` 结构体地址

events: `EPOLLIN`、`EPOLLOUT`、`EPOLLERR`

data: 联合体

int fd: 对应监听事件的fd

void *ptr: 泛型指针, 可以指向任何类型, 所以说可以指向一个结构体, 结构体里定义回调函数和对应监听事件的fd

uint32_t u32

uint32_t u64

原来: socket、bind、listen -- epoll_create 创建监听 红黑树 -- 返回 epfd -- epoll_ctl() 向树上添加一个监听fd -- while (1) --
-- epoll_wait 监听 -- 对应监听fd有事件产生 -- 返回 监听满足数组。 -- 判断返回数组元素 -- lfd满足 -- Accept -- cfd 满足
-- read() --- 小->大 -- write回去。

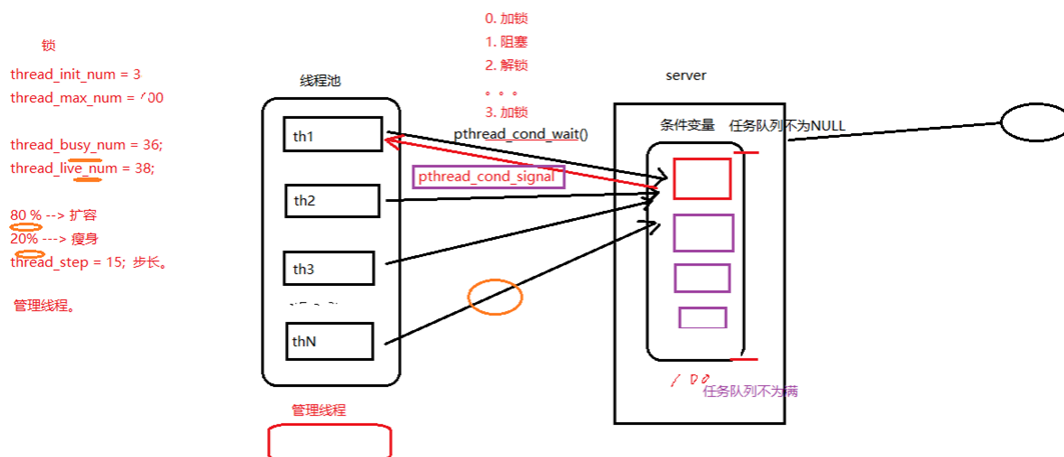
反应堆: 不但要监听 cfd 的读事件、还要监听cfd的写事件。

socket、bind、listen -- epoll_create 创建监听 红黑树 -- 返回 epfd -- epoll_ctl() 向树上添加一个监听fd -- while (1) --
-- epoll_wait 监听 -- 对应监听fd有事件产生 -- 返回 监听满足数组。 -- 判断返回数组元素 -- lfd满足 -- Accept -- cfd 满足
-- read() --- 小->大 -- cfd从监听红黑树上摘下 -- EPOLLOUT -- 回调函数 -- epoll_ctl() -- EPOLL_CTL_ADD 重新放到红黑上监听写事件
-- 等待 epoll_wait 返回 -- 说明 cfd 可写 -- write回去 -- cfd从监听红黑树上摘下 -- EPOLLIN
-- epoll_ctl() -- EPOLL_CTL_ADD 重新放到红黑上监听读事件 -- epoll_wait 监听

反应堆的理解: 加入IO转接之后, 有了事件, server才去处理, 这里反应堆也是这样, 由于网络环境复杂, 服务器处理数据之后, 可能并不能直接写回去, 比如遇到网络繁忙或者对方缓冲区已经满了这种情况, 就不能直接写回给客户端。反应堆就是在处理数据之后, 监听写事件, 能写回客户端了, 才去做写回操作。写回之后, 在改回监听读事件, 以此循环

线程池

模型原理分析



线程池的组成主要分为 3 个部分, 这三部分配合工作就可以得到一个完整的线程池:

1. 任务队列, 存储需要处理的任务, 由工作的线程来处理这些任务

通过线程池提供的 API 函数, 将一个待处理的任务添加到任务队列, 或者从任务队列中删除

已处理的任务会被从任务队列中删除

线程池的使用者, 也就是调用线程池函数往任务队列中添加任务的线程就是生产者线程

2. 工作的线程 (任务队列任务的消费者), N个

线程池中维护了一定数量的工作线程，他们的作用是是不停的读任务队列，从里边取出任务并处理

工作的线程相当于是任务队列的消费者角色，

如果任务队列为空，工作的线程将会被阻塞 (使用条件变量 / 信号量阻塞)

如果阻塞之后有了新的任务，由生产者将阻塞解除，工作线程开始工作

3. 管理者线程（不处理任务队列中的任务），1个

它的任务是周期性的对任务队列中的任务数量以及处于忙状态的工作线程个数进行检测

当任务过多的时候，可以适当的创建一些新的工作线程

当任务过少的时候，可以适当的销毁一些工作的线程

代码实现

```
// 任务结构体
typedef struct Task
{
    void (*function)(void* arg);
    void* arg;
}Task;
```

```
// 线程池结构体
struct ThreadPool
{
    // 任务队列
    Task* taskQ;
    int queueCapacity; // 容量
    int queueSize;     // 当前任务个数
    int queueFront;    // 队头 -> 取数据
    int queueRear;     // 队尾 -> 放数据

    pthread_t managerID; // 管理者线程ID
    pthread_t *threadIDs; // 工作的线程ID
    int minNum;          // 最小线程数量
    int maxNum;          // 最大线程数量
    int busyNum;         // 忙的线程的个数
    int liveNum;         // 存活的线程的个数
    int exitNum;         // 要销毁的线程个数
    pthread_mutex_t mutexPool; // 锁整个的线程池
    pthread_mutex_t mutexBusy; // 锁busyNum变量
    pthread_cond_t notFull;    // 任务队列是不是满了
    pthread_cond_t notEmpty;   // 任务队列是不是空了

    int shutdown; // 是不是要销毁线程池，销毁为1，不销毁为0
};
```

```

#ifndef _THREADPOOL_H
#define _THREADPOOL_H

typedef struct ThreadPool ThreadPool;
// 创建线程池并初始化
ThreadPool *threadPoolCreate(int min, int max, int queueSize);

// 销毁线程池
int threadPoolDestroy(ThreadPool* pool);

// 给线程池添加任务
void threadPoolAdd(ThreadPool* pool, void(*func)(void*), void* arg);

// 获取线程池中工作的线程的个数
int threadPoolBusyNum(ThreadPool* pool);

// 获取线程池中活着的线程的个数
int threadPoolAliveNum(ThreadPool* pool);

//////////
// 工作的线程(消费者线程)任务函数
void* worker(void* arg);
// 管理者线程任务函数
void* manager(void* arg);
// 单个线程退出
void threadExit(ThreadPool* pool);
#endif // _THREADPOOL_H

```

```

ThreadPool* threadPoolCreate(int min, int max, int queueSize)
{
    ThreadPool* pool = (ThreadPool*)malloc(sizeof(ThreadPool));
    do
    {
        if (pool == NULL)
        {
            printf("malloc threadpool fail...\n");
            break;
        }

        pool->threadIDs = (pthread_t*)malloc(sizeof(pthread_t) * max);
        if (pool->threadIDs == NULL)
        {
            printf("malloc threadIDs fail...\n");
            break;
        }
        memset(pool->threadIDs, 0, sizeof(pthread_t) * max);
        pool->minNum = min;
        pool->maxNum = max;
        pool->busyNum = 0;
        pool->liveNum = min;    // 和最小个数相等
    } while(0);
}

```

```
pool->exitNum = 0;

if (pthread_mutex_init(&pool->mutexPool, NULL) != 0 ||
    pthread_mutex_init(&pool->mutexBusy, NULL) != 0 ||
    pthread_cond_init(&pool->notEmpty, NULL) != 0 ||
    pthread_cond_init(&pool->notFull, NULL) != 0)
{
    printf("mutex or condition init fail...\n");
    break;
}

// 任务队列
pool->taskQ = (Task*)malloc(sizeof(Task) * queueSize);
pool->queueCapacity = queueSize;
pool->queueSize = 0;
pool->queueFront = 0;
pool->queueRear = 0;

pool->shutdown = 0;

// 创建线程
pthread_create(&pool->managerID, NULL, manager, pool);
for (int i = 0; i < min; ++i)
{
    pthread_create(&pool->threadIDs[i], NULL, worker, pool);
}
return pool;
} while (0);

// 释放资源
if (pool && pool->threadIDs) free(pool->threadIDs);
if (pool && pool->taskQ) free(pool->taskQ);
if (pool) free(pool);

return NULL;
}

int threadPoolDestroy(ThreadPool* pool)
{
    if (pool == NULL)
    {
        return -1;
    }

    // 关闭线程池
    pool->shutdown = 1;
    // 阻塞回收管理者线程
    pthread_join(pool->managerID, NULL);
    // 唤醒阻塞的消费者线程
    for (int i = 0; i < pool->liveNum; ++i)
    {
        pthread_cond_signal(&pool->notEmpty);
    }
    // 释放堆内存
```

```
    if (pool->taskQ)
    {
        free(pool->taskQ);
    }
    if (pool->threadIDs)
    {
        free(pool->threadIDs);
    }

    pthread_mutex_destroy(&pool->mutexPool);
    pthread_mutex_destroy(&pool->mutexBusy);
    pthread_cond_destroy(&pool->notEmpty);
    pthread_cond_destroy(&pool->notFull);

    free(pool);
    pool = NULL;

    return 0;
}

void threadPoolAdd(ThreadPool* pool, void(*func)(void*), void* arg)
{
    pthread_mutex_lock(&pool->mutexPool);
    while (pool->queueSize == pool->queueCapacity && !pool->shutdown)
    {
        // 阻塞生产者线程
        pthread_cond_wait(&pool->notFull, &pool->mutexPool);
    }
    if (pool->shutdown)
    {
        pthread_mutex_unlock(&pool->mutexPool);
        return;
    }
    // 添加任务
    pool->taskQ[pool->queueRear].function = func;
    pool->taskQ[pool->queueRear].arg = arg;
    pool->queueRear = (pool->queueRear + 1) % pool->queueCapacity;
    pool->queueSize++;

    pthread_cond_signal(&pool->notEmpty);
    pthread_mutex_unlock(&pool->mutexPool);
}

int threadPoolBusyNum(ThreadPool* pool)
{
    pthread_mutex_lock(&pool->mutexBusy);
    int busyNum = pool->busyNum;
    pthread_mutex_unlock(&pool->mutexBusy);
    return busyNum;
}

int threadPoolAliveNum(ThreadPool* pool)
{

```

```
pthread_mutex_lock(&pool->mutexPool);
int aliveNum = pool->liveNum;
pthread_mutex_unlock(&pool->mutexPool);
return aliveNum;
}

void* worker(void* arg)
{
    ThreadPool* pool = (ThreadPool*)arg;

    while (1)
    {
        pthread_mutex_lock(&pool->mutexPool);
        // 当前任务队列是否为空
        while (pool->queueSize == 0 && !pool->shutdown)
        {
            // 阻塞工作线程
            pthread_cond_wait(&pool->notEmpty, &pool->mutexPool);

            // 判断是不是要销毁线程
            if (pool->exitNum > 0)
            {
                pool->exitNum--;
                if (pool->liveNum > pool->minNum)
                {
                    pool->liveNum--;
                    pthread_mutex_unlock(&pool->mutexPool);
                    threadExit(pool);
                }
            }
        }

        // 判断线程池是否被关闭了
        if (pool->shutdown)
        {
            pthread_mutex_unlock(&pool->mutexPool);
            threadExit(pool);
        }

        // 从任务队列中取出一个任务
        Task task;
        task.function = pool->taskQ[pool->queueFront].function;
        task.arg = pool->taskQ[pool->queueFront].arg;
        // 移动头结点
        pool->queueFront = (pool->queueFront + 1) % pool->queueCapacity;
        pool->queueSize--;
        // 解锁
        pthread_cond_signal(&pool->notEmpty);
        pthread_mutex_unlock(&pool->mutexPool);

        printf("thread %ld start working...\n", pthread_self());
        pthread_mutex_lock(&pool->mutexBusy);
        pool->busyNum++;
        pthread_mutex_unlock(&pool->mutexBusy);
    }
}
```

```

        task.function(task.arg);
        free(task.arg);
        task.arg = NULL;

        printf("thread %ld end working...\n", pthread_self());
        pthread_mutex_lock(&pool->mutexBusy);
        pool->busyNum--;
        pthread_mutex_unlock(&pool->mutexBusy);
    }
    return NULL;
}

void* manager(void* arg)
{
    ThreadPool* pool = (ThreadPool*)arg;
    while (!pool->shutdown)
    {
        // 每隔3s检测一次
        sleep(3);

        // 取出线程池中任务的数量和当前线程的数量
        pthread_mutex_lock(&pool->mutexPool);
        int queueSize = pool->queueSize;
        int liveNum = pool->liveNum;
        pthread_mutex_unlock(&pool->mutexPool);

        // 取出忙的线程的数量
        pthread_mutex_lock(&pool->mutexBusy);
        int busyNum = pool->busyNum;
        pthread_mutex_unlock(&pool->mutexBusy);

        // 添加线程
        // 任务的个数>存活的线程个数 && 存活的线程数<最大线程数
        if (queueSize > liveNum && liveNum < pool->maxNum)
        {
            pthread_mutex_lock(&pool->mutexPool);
            int counter = 0;
            for (int i = 0; i < pool->maxNum && counter < NUMBER
                && pool->liveNum < pool->maxNum; ++i)
            {
                if (pool->threadIDs[i] == 0)
                {
                    pthread_create(&pool->threadIDs[i], NULL, worker, pool);
                    counter++;
                    pool->liveNum++;
                }
            }
            pthread_mutex_unlock(&pool->mutexPool);
        }

        // 销毁线程
        // 忙的线程*2 < 存活的线程数 && 存活的线程>最小线程数
        if (busyNum * 2 < liveNum && liveNum > pool->minNum)
        {
            pthread_mutex_lock(&pool->mutexPool);

```

```

        pool->exitNum = NUMBER;
        pthread_mutex_unlock(&pool->mutexPool);
        // 让工作的线程自杀
        for (int i = 0; i < NUMBER; ++i)
        {
            pthread_cond_signal(&pool->notEmpty);
        }
    }
}
return NULL;
}

void threadExit(ThreadPool* pool)
{
    pthread_t tid = pthread_self();
    for (int i = 0; i < pool->maxNum; ++i)
    {
        if (pool->threadIDs[i] == tid)
        {
            pool->threadIDs[i] = 0;
            printf("threadExit() called, %ld exiting...\n", tid);
            break;
        }
    }
    pthread_exit(NULL);
}

```

```

void taskFunc(void* arg)
{
    int num = *(int*)arg;
    printf("thread %ld is working, number = %d\n",
        pthread_self(), num);
    sleep(1);
}

int main()
{
    // 创建线程池
    ThreadPool* pool = threadPoolCreate(3, 10, 100);
    for (int i = 0; i < 100; ++i)
    {
        int* num = (int*)malloc(sizeof(int));
        *num = i + 100;
        threadPoolAdd(pool, taskFunc, num);
    }

    sleep(30);

    threadPoolDestroy(pool);
    return 0;
}

```

C++版本

手写线程池 - C改C++版

UDP通信

TCP和UDP通信优缺点

TCP: 面向连接的, 可靠数据包传输。对于不稳定的网络层, 采取完全弥补的通信方式。丢包重传。

优点: 稳定。
数据流量稳定、速度稳定、顺序

缺点: 传输速度慢。相率低。开销大。

使用场景: 数据的完整型要求较高, 不追求效率。
大数据传输、文件传输。

UDP: 无连接的, 不可靠的数据报传递。对于不稳定的网络层, 采取完全不弥补的通信方式。默认还原网络状况

优点: | I
传输速度快。相率高。开销小。

缺点: 不稳定。
数据流量。速度。顺序。

使用场景: 对时效性要求较高场合。稳定性其次。

游戏、视频会议、视频电话。

腾讯、华为、阿里 --- 应用层数据校验协议, 弥补udp的不足。

UDP的通信流程

相对TCP来说, accept()和connect()被舍弃;

recv()和send()只能用于TCP;

serve端:

1. lfd = socket(AF_INET, SOCK_DGRAM, 0) 由SOCK_STREAM 改为了 SOCK_DGRAM
2. bind(lfd, 地址结构, 地址结构大小)
3. listen() -- 可有可无
4. while(1){ read(cfd, buf, sizeof)-->被替换为-->recvfrom() write()-->被替换为-->sendto() }
5. close()

client端:

1. confd = socket(AF_INET, SOCK_DGRAM, 0)
2. sendto(服务器的地址结构, 地址结构大小)
3. recvfrom()读回新数据
4. close()

recvfrom和sendto

```
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr,
socklen_t *addrlen);
```

sockfd: 套接字

buf: 缓冲区地址

len: 缓冲区大小

flags: 0

src_addr: (struct sockaddr *) &addr 传出。对端的地址结构

addrlen: 传入传出。

返回值: 成功接收数据字节数。失败: -1 errno。0: 对端关闭

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr
* dest_addr, socklen_t addrlen);
```

sockfd: 套接字

buf: 存储数据的缓冲区

len: 数据长度

flags: 0

src_addr: (struct sockaddr *) &addr 传入。目标地址结构

addrlen: 地址结构长度。

返回值: 成功写出数据字节数。失败 -1 error

代码实现

```
1. #include <string.h>
2. #include <stdio.h>
3. #include <unistd.h>
4. #include <arpa/inet.h>
5. #include <ctype.h>
6.
7. #define SERV_PORT 8000
8.
9. int main(void)
10. {
11.     struct sockaddr_in serv_addr, clie_addr;
12.     socklen_t clie_addr_len;
13.     int sockfd;
14.     char buf[BUFSIZ];
15.     char str[INET_ADDRSTRLEN];
16.     int i, n;
17.
```

```
18.     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
19.
20.     bzero(&serv_addr, sizeof(serv_addr));
21.     serv_addr.sin_family = AF_INET;
22.     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
23.     serv_addr.sin_port = htons(SERV_PORT);
24.
25.     bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
26.
27.     printf("Accepting connections ...\n");
28.     while (1) {
29.         clie_addr_len = sizeof(clie_addr);
30.         n = recvfrom(sockfd, buf, BUFSIZ, 0, (struct sockaddr *)&clie_addr,
&clie_addr_len);
31.         if (n == -1)
32.             perror("recvfrom error");
33.
34.         printf("received from %s at PORT %d\n",
35.             inet_ntop(AF_INET, &clie_addr.sin_addr, str, sizeof(str)),
36.             ntohs(clie_addr.sin_port));
37.
38.         for (i = 0; i < n; i++)
39.             buf[i] = toupper(buf[i]);
40.
41.         n = sendto(sockfd, buf, n, 0, (struct sockaddr *)&clie_addr,
sizeof(clie_addr));
42.         if (n == -1)
43.             perror("sendto error");
44.     }
45.
46.     close(sockfd);
47.
48.     return 0;
49. }
```

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  #include <unistd.h>
4.  #include <arpa/inet.h>
5.  #include <ctype.h>
6.
7.  #define SERV_PORT 8000
8.
9.  int main(int argc, char *argv[])
10. {
11.     struct sockaddr_in servaddr;
12.     int sockfd, n;
13.     char buf[BUFSIZ];
14.
15.     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

```

16.
17.     bzero(&servaddr, sizeof(servaddr));
18.     servaddr.sin_family = AF_INET;
19.     inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);
20.     servaddr.sin_port = htons(SERV_PORT);
21.
22.     bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
23.
24.     while (fgets(buf, BUFSIZ, stdin) != NULL) {
25.         n = sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr *)&servaddr,
sizeof(servaddr));
26.         if (n == -1)
27.             perror("sendto error");
28.
29.         n = recvfrom(sockfd, buf, BUFSIZ, 0, NULL, 0);           //NULL:不关心对
端信息
30.         if (n == -1)
31.             perror("recvfrom error");
32.
33.         write(STDOUT_FILENO, buf, n);
34.     }
35.
36.     close(sockfd);
37.
38.     return 0;
39. }

```

本地套接字通信

IPC: pipe、fifo、mmap、信号、本地套 (domain) --- CS模型

对比网络编程 TCP C/S模型, 注意以下几点:

1. int socket(int domain, int type, int protocol); 参数 domain: AF_INET --> AF_UNIX/AF_LOCAL

type: SOCK_STREAM/SOCK_DGRAM 都可以。

2. 地址结构: sockaddr_in --> sockaddr_un

- struct sockaddr_in srv_addr; --> struct sockaddr_un srv_addr;
- srv_addr.sin_family = AF_INET; --> srv_addr.sun_family = AF_UNIX;
- srv_addr.sin_port = htons(8888); strcpy(srv_addr.sun_path, "srv.socket")
- srv_addr.sin_addr.s_addr = htonl(INADDR_ANY); len = offsetof(struct sockaddr_un, sun_path) + strlen("srv.socket");

bind(fd, (struct sockaddr *)&srv_addr, sizeof(srv_addr)); --> bind(fd, (struct sockaddr *)&srv_addr, len);

3. bind()函数调用成功, 会创建一个 socket。因此为保证bind成功, 通常我们在 bind之前, 可以使用 unlink("srv.socket");

4. 客户端不能依赖“隐式绑定”。并且应该在通信建立过程中, 创建且初始化2个地址结构:

- client_addr --> bind()

- server_addr --> connect();

```

1. #include <stdio.h>
2. #include <unistd.h>
3. #include <sys/socket.h>
4. #include <strings.h>
5. #include <string.h>
6. #include <ctype.h>
7. #include <arpa/inet.h>
8. #include <sys/un.h>
9. #include <stddef.h>
10.
11. #include "wrap.h"
12.
13. #define SERV_ADDR "serv.sock"
14.
15. int main(void)
16. {
17.     int lfd, cfd, len, size, i;
18.     struct sockaddr_un servaddr, cliaddr;
19.     char buf[4096];
20.
21.     lfd = Socket(AF_UNIX, SOCK_STREAM, 0);
22.
23.     bzero(&servaddr, sizeof(servaddr));
24.     servaddr.sun_family = AF_UNIX;
25.     strcpy(servaddr.sun_path, SERV_ADDR);
26.
27.     len = offsetof(struct sockaddr_un, sun_path) + strlen(servaddr.sun_path);
/* servaddr total len */
28.
29.     unlink(SERV_ADDR); /* 确保bind之前serv.sock文
件不存在,bind会创建该文件 */
30.     Bind(lfd, (struct sockaddr *)&servaddr, len); /* 参3不能是
sizeof(servaddr) */
31.
32.     Listen(lfd, 20);
33.
34.     printf("Accept ...\n");
35.     while (1) {
36.         len = sizeof(cliaddr); //AF_UNIX大小+108B
37.
38.         cfd = Accept(lfd, (struct sockaddr *)&cliaddr, (socklen_t *)&len);
39.
40.         len -= offsetof(struct sockaddr_un, sun_path); /* 得到文件名的长度
*/
41.         cliaddr.sun_path[len] = '\0'; /* 确保打印时,没有
乱码出现 */
42.
43.         printf("client bind filename %s\n", cliaddr.sun_path);
44.
45.         while ((size = read(cfd, buf, sizeof(buf))) > 0) {
46.             for (i = 0; i < size; i++)

```

```
47.         buf[i] = toupper(buf[i]);
48.         write(cfd, buf, size);
49.     }
50.     close(cfd);
51. }
52. close(lfd);
53.
54. return 0;
55. }
```

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <sys/types.h>
4. #include <sys/socket.h>
5. #include <strings.h>
6. #include <string.h>
7. #include <ctype.h>
8. #include <arpa/inet.h>
9. #include <sys/un.h>
10. #include <stddef.h>
11.
12. #include "wrap.h"
13.
14. #define SERV_ADDR "serv.socket"
15. #define CLIE_ADDR "clie.socket"
16.
17. int main(void)
18. {
19.     int cfd, len;
20.     struct sockaddr_un servaddr, cliaddr;
21.     char buf[4096];
22.
23.     cfd = Socket(AF_UNIX, SOCK_STREAM, 0);
24.
25.     bzero(&cliaddr, sizeof(cliaddr));
26.     cliaddr.sun_family = AF_UNIX;
27.     strcpy(cliaddr.sun_path, CLIE_ADDR);
28.
29.     len = offsetof(struct sockaddr_un, sun_path) + strlen(cliaddr.sun_path);
/* 计算客户端地址结构有效长度 */
30.
31.     unlink(CLIE_ADDR);
32.     Bind(cfd, (struct sockaddr *)&cliaddr, len);
/* 客户端也需要bind, 不能依赖自动绑定 */
33.
34.
35.     bzero(&servaddr, sizeof(servaddr));
/* 构造server 地址 */
36.     servaddr.sun_family = AF_UNIX;
37.     strcpy(servaddr.sun_path, SERV_ADDR);
```

```
38.
39.     len = offsetof(struct sockaddr_un, sun_path) + strlen(servaddr.sun_path);
/* 计算服务器端地址结构有效长度 */
40.
41.     Connect(cfd, (struct sockaddr *)&servaddr, len);
42.
43.     while (fgets(buf, sizeof(buf), stdin) != NULL) {
44.         write(cfd, buf, strlen(buf));
45.         len = read(cfd, buf, sizeof(buf));
46.         write(STDOUT_FILENO, buf, len);
47.     }
48.
49.     close(cfd);
50.
51.     return 0;
52. }
```

通信总结

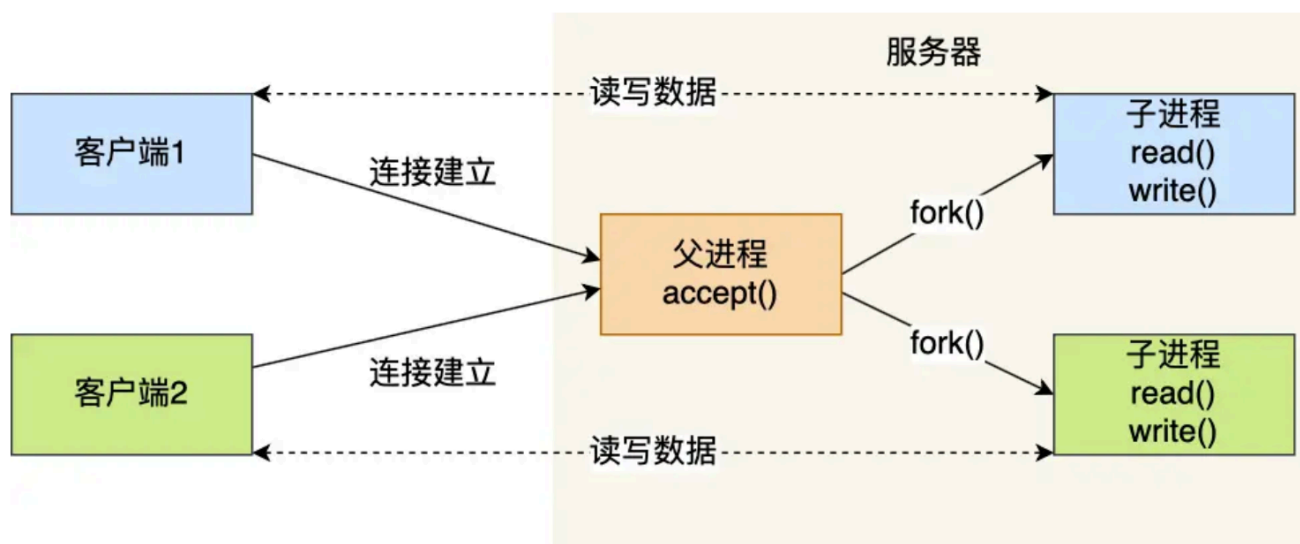
Linux网络编程本质就是实现服务端和客户端的通信传输，而Linux系统编程也有讲过本地进程间的通信，所以总结一下各种通信的方式，以免混淆；

本地通信：

1. pipe(管道)：实现了有血缘关系进程间的通信
2. fifo：可用于非血缘关系进程间的通信
3. mmap内存映射：上面两个的功能它都可以实现
4. 信号：信号量
5. 本地套接字：socket(),参数 domain: AF_UNIX/AF_LOCAL

网络通信：

1. 多进程：也就是为每个客户端分配一个进程来处理请求(记得回收子进程)



2. 多线程：当服务器与客户端 TCP 完成连接后，通过 `pthread_create()` 函数创建线程，然后将「已连接 Socket」的文件描述符传递给线程函数，接着在线程里和客户端进行通信，从而达到并发处理的目的。

为每个请求分配一个进程/线程的方式不合适，那有没有可能只使用一个进程来维护多个 Socket 呢？答案是有的，那就是 **I/O 多路复用** 技术：

3. select/poll：存在缺点--当客户端越多，也就是 Socket 集合越大，Socket 集合的遍历和拷贝会带来很大的开销，因此也很难应对 C10K。
4. epoll：很好的解决了 select/poll 的问题
 - epoll 在内核里使用「红黑树」来关注进程所有待检测的 Socket，红黑树是个高效的数据结构，增删改一般时间复杂度是 $O(\log n)$ ，通过对这棵黑红树的管理，不需要像 select/poll 在每次操作时都传入整个 Socket 集合，减少了内核和用户空间大量的数据拷贝和内存分配。
 - epoll 使用事件驱动的机制，内核里维护了一个「链表」来记录就绪事件，只将有事件发生的 Socket 集合传递给应用程序，不需要像 select/poll 那样轮询扫描整个集合（包含有和无事件的 Socket），大大提高了检测的效率。

Libevent库

简介

libevent的特点：开源、精简、跨平台、专注于通信（本地和网络都可以）

安装：

1. 进入官网下载安装包后拖入虚拟机，压缩包名为 libevent-2.1.11-stable.tar.gz
2. 解压：

```
tar -zxvf libevent-2.1.11-stable.tar.gz
```

3. `./configure`
4. `make`
5. `sudo make install`
6. `cd sample`，里面有demo 可以检测是否安装成功
7. 编译使用库的.c文件时，需要加 `-levent`

常规event

1 创建 event_base (乐高底座)

```
struct event_base *event_base_new(void);
struct event_base *base = event_base_new();
```

2 创建 事件event

```
常规事件 event --> event_new();
bufferevent --> bufferevent_socket_new();
```

3 将事件 添加到 base上

```
int event_add(struct event *ev, const struct timeval *tv)
```

4 循环监听事件满足

```
int event_base_dispatch(struct event_base *base);
event_base_dispatch(base);
```

5 释放 event_base

```
event_base_free(base);
```

1. 创建底座: `struct event_base *base = event_base_new()/bufferevent_socket_new();`
2. 创建事件: `<font style="color:rgb(0, 0, 0);">struct event *event_new(struct event_base *base, evutil_socket_t fd, short what, event_callback_fn callback, void *arg)</font>`

base: event_base_new的返回值, 也就是底座

fd: 绑定到事件上的文件描述符

what: 监听的事件(EV_READ、EV_WRITE、EV_PERSIST(持续触发))

callback: 一旦事件满足监听条件, 回调的函数

typedef void (*event_callback_fn函数名)(evutil_socket_t fd, short what , void *arg)

arg: 回调函数的参数

返回值: 成功--返回创建的event,

3. 添加事件到底座base上: `int event_add(struct event *ev, const struct timeval *tv);`

ev: event_new的返回值

tv: 为NULL: 一直等到事件被触发 回调函数会被调用; 为非0: 没有事件触发, 时间到了, 回调函数依旧被调用

4. 启动循环: `int event_base_dispatch(struct event_base *base);` 内部就是while(1){epoll}
5. 释放事件: `<font style="color:rgb(0, 0, 0);">int event_free(struct event *ev);</font>`

ev: event_new的返回值

event实现本地通信

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. #include <sys/types.h>
5. #include <sys/stat.h>
6. #include <string.h>
7. #include <fcntl.h>
8. #include <event2/event.h>
9.
10. // 对操作处理函数
11. void read_cb(evutil_socket_t fd, short what, void *arg)
12. {
13.     // 读管道
14.     char buf[1024] = {0};
15.
16.     int len = read(fd, buf, sizeof(buf));
17.
18.     printf("read event: %s \n", what & EV_READ ? "Yes" : "No");
19.     printf("data len = %d, buf = %s\n", len, buf);
20.
21.     sleep(1);
22. }
23.
24.
25.
26. int main(int argc, const char* argv[])
27. {
28.     unlink("myfifo");
29.
30.     //创建有名管道
31.     mkfifo("myfifo", 0664);
32.
33.     // open file
34.     //int fd = open("myfifo", O_RDONLY | O_NONBLOCK);
35.     int fd = open("myfifo", O_RDONLY);
36.     if(fd == -1)
37.     {
38.         perror("open error");
39.         exit(1);
40.     }
41.
42.     // 创建个event_base
43.     struct event_base* base = event_base_new() ;
44.
45.
46.     // 创建事件
47.     struct event *ev = NULL;
48.     ev = event_new(base, fd, EV_READ | EV_PERSIST, read_cb, NULL);
49.
50.     // 添加事件
```

```
51.     event_add(ev, NULL);
52.
53.     // 事件循环
54.     event_base_dispatch(base); // while (1) { epoll();}
55.
56.     // 释放资源
57.     event_free(ev);
58.     event_base_free(base);
59.     close(fd);
60.
61.     return 0;
62. }
```

```
1.  #include <stdio.h>
2.  #include <unistd.h>
3.  #include <stdlib.h>
4.  #include <sys/types.h>
5.  #include <sys/stat.h>
6.  #include <string.h>
7.  #include <fcntl.h>
8.  #include <event2/event.h>
9.
10. // 对操作处理函数
11. void write_cb(evutil_socket_t fd, short what, void *arg)
12. {
13.     // write管道
14.     char buf[1024] = {0};
15.
16.     static int num = 0;
17.     sprintf(buf, "hello,world-%d\n", num++);
18.     write(fd, buf, strlen(buf)+1);
19.
20.     sleep(1);
21. }
22.
23.
24. // 写管道
25. int main(int argc, const char* argv[])
26. {
27.     // open file
28.     //int fd = open("myfifo", O_WRONLY | O_NONBLOCK);
29.     int fd = open("myfifo", O_WRONLY);
30.     if(fd == -1)
31.     {
32.         perror("open error");
33.         exit(1);
34.     }
35.
36.     // 写管道
37.     struct event_base* base = NULL;
38.     base = event_base_new();
```

```

39.
40. // 创建事件
41. struct event* ev = NULL;
42. // 检测的写缓冲区是否有空间写
43. //ev = event_new(base, fd, EV_WRITE , write_cb, NULL);
44. ev = event_new(base, fd, EV_WRITE | EV_PERSIST, write_cb, NULL);
45.
46. // 添加事件
47. event_add(ev, NULL);
48.
49. // 事件循环
50. event_base_dispatch(base);
51.
52. // 释放资源
53. event_free(ev);
54. event_base_free(base);
55. close(fd);
56.
57. return 0;
58. }

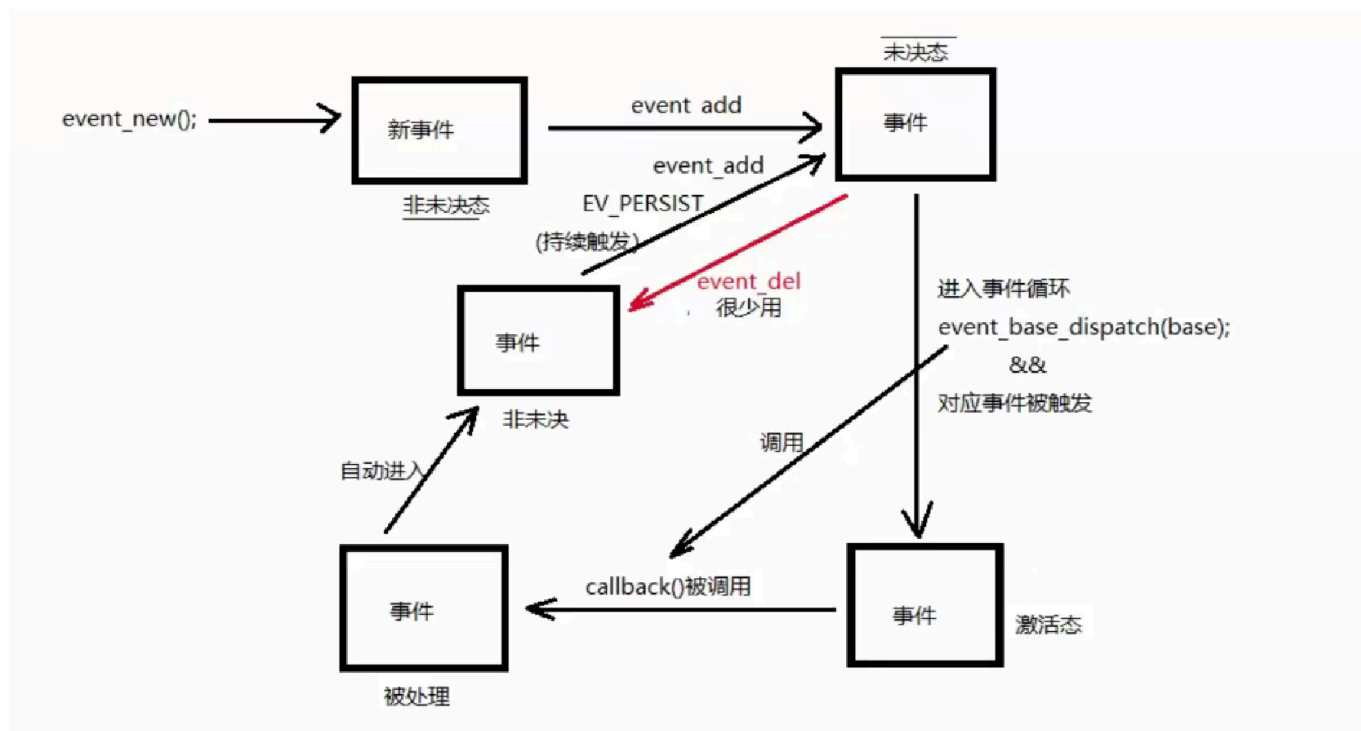
```

如果我的写端先关闭，读端只要是持续触发状态(EV_PERSIST)就会一直读，而不管对端有没有发过来数据

事件的未决和非未决

未决：有资格被处理，但还没有被处理

非未决：没有资格被处理



bufferevent(有缓冲区)

头文件:

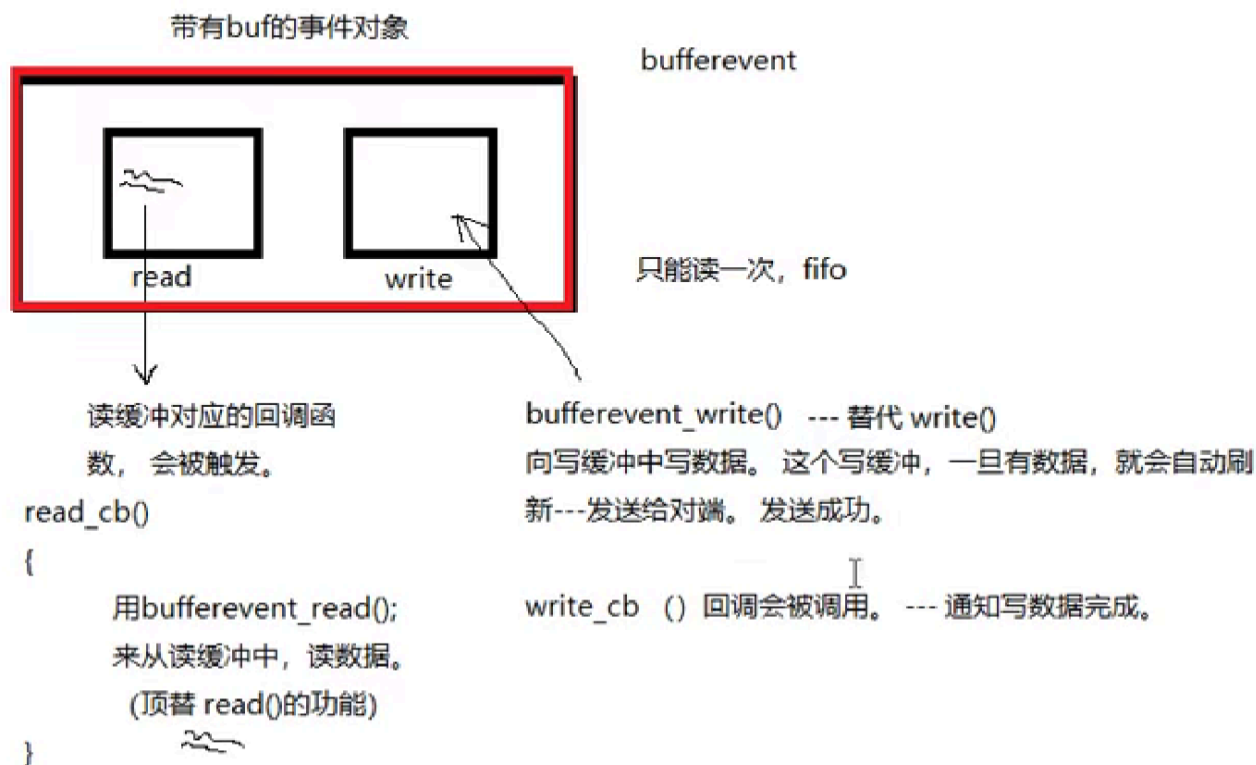
```
#include <event2/bufferevent.h>
```

原理:

bufferevent有两个缓冲区: 也是队列实现, 数据只能读一次, 先进先出(管道)

读: 有数据-->读回函数被调用-->使用bufferevent_read()-->读数据

写: 使用bufferevent_write()-->向写缓冲区中写数据-->写完后回调函数被调用(通知写数据成功了,比较鸡肋)



bufferevent的创建和释放

1. 创建:

```
struct bufferevent *bev = bufferevent_socket_new(base, fd, BEV_OPT_CLOSE_ON_FREE)
```

```
struct bufferevent *bufferevent_socket_new(struct event_base *base, evutil_socket_t fd,
enum bufferevent_options options);
```

base: event_base (底座)

fd: 文件描述符, 此处有歧义: **应该传入的是与客户端连接的套接字描述符 cfd, 而不是用于监听的套接字描述符 lfd。**

options: BEV_OPT_CLOSE_ON_FREE

返回: 成功创建的 bufferevent事件对象

2. 释放:

```
void bufferevent_socket_free(struct bufferevent *bev);
```

bev: bufferevent_socket_new的返回值 (bufferevent事件对象)

给bufferevent读写缓冲区设置回调

1. 设置回调****:

```
void bufferevent_setcb(struct bufferevent * bufev,
```

```
    bufferevent_data_cb readcb,

    bufferevent_data_cb writecb,

    bufferevent_event_cb eventcb,

    void *cbarg );
```

bufev: bufferevent_socket_new的返回值 (bufferevent事件对象)

readcb: 设置bufferevent读缓冲的对应回调 read_cb { bufferevent_read() 读数据 }

writecb: 设置bufferevent写缓冲的对应回调 write_cb {} --> 给调用者发送写成功的通知, 可传NULL

eventcb: 设置事件回调, 也可传NULL

cbarg: 上述回调函数的参数

2. read_cb:

read 回调函数类型:

```
typedef void (*bufferevent_data_cb)(struct bufferevent *bev, void*ctx);
void read_cb (struct bufferevent *bev, void *cbarg )
{
    .....
    bufferevent_read();
}
```

bufferevent_read()函数的原型:

```
size_t bufferevent_read(struct bufferevent *bev, void *buf, size_t bufsiz);
```

3. write_cb:

```
int bufferevent_write(struct bufferevent *bufev, const void *data, size_t size);
```

4. event_cb:

eventcb: 设置 事件回调。 也可传NULL

```
typedef void (*bufferevent_event_cb)(struct bufferevent *bev, short events, void *ctx);
void event_cb(struct bufferevent *bev, short events, void *ctx)
{
    * * * * *
}
events: BEV_EVENT_CONNECTED
```

禁用、启用缓冲区

默认：新建的 bufferevent 写缓冲是 enable 的。而，读缓冲是 **disable** 的。

void bufferevent_enable(struct bufferevent *bufev, short events); 启用缓冲区

通常用来启用 bufferevent 的 read 缓冲。

void bufferevent_disable(struct bufferevent *bufev, short events); 禁用

events : EV_READ、EV_WRITE、EV_READ|EV_WRITE

short bufferevent_get_enabled(struct bufferevent *bufev);

获取缓冲区的禁用状态，需要借助 & 来得到。

通信相关函数

客户端：

socket(); connect();

int bufferevent_socket_connect(struct bufferevent *bev, struct sockaddr *address, int addrlen);

bev: bufferevent 事件对象（封装了fd）

address、len：等同于 connect()的参2和参3

服务器：

加头文件：#include <event2/listener.h>

这一个函数可以完成：socket()、bind()、listen()、accept() 这四个函数的作用

struct evconnlistener * listener = evconnlistener_new_bind()

struct evconnlistener *evconnlistener_new_bind (

```
struct event_base *base,  
  
evconnlistener_cb cb,  
  
void *ptr,  
  
unsigned flags,  
  
int backlog,  
  
const struct sockaddr *sa,  
  
int socklen);
```

base: event_base;

cb: 回调函数-->一旦被回调,说明在其内部应该与客户端完成数据读写操作 进行通信;

ptr: 回调函数的参数,可以将base传进去,在回调函数中bufferevent_socket_new时要用

flags: LEV_OPT_CLOSE_ON_FREE | LEV_OPT_REUSEABLE

backlog: listen()的参数二,传-1表示最大值

sa: 服务器自己的地址结构体

socklen: 服务器自己地址结构体的大小

返回值: 成功创建的监听器

释放:

```
void evconnlistener_free(struct evconnlistener *lev)
```

回调函数:

```
void listener_cb(struct evconnlistener *listener,  
                 evutil_socket_t fd,  
                 struct sockaddr *addr,  
                 int len, void *ptr)  
{  
    .....  
}
```

listener：evconnlistener_new_bind 函数的返回值。

sock：用于通信的文件描述符。

addr：客户端的 IP+端口。

len：addr 的 len。

ptr：外部 ptr 传递进来值。

通信流程分析

服务器：

1. 创建event_base（底座）
2. 创建服务器连接监听器 evconnlistener_new_bind()；它内部会监听客户端的连接，有的话就调用回调
3. 在evconnlistener_new_bind()的回调函数中，处理接受与客户端连接后的操作
4. 监听器回调函数被调用，说明有一个新客户端连接上来，会得到一个新的cfd，用于跟客户端通信
5. 在监听器的回调中使用 bufferevent_socket_new() 创建一个新 bufferevent事件，将cfd封装到这个事件对象中
6. 在监听器的回调中使用 bufferevent_setcb() 给这个事件对象的 read、write、event设置回调
7. 在监听器的回调中设置 bufferevent的读写缓冲区 enable/disable
8. 接受、发送数据 bufferevent_read()/bufferevent_write()-->在bufferevent的读写回调中进行
9. 启动循环监听 event_base_dispatch(base)
10. 释放资源 free

客户端：

1. 创建 event_base。
2. 使用 bufferevent_socket_new() 创建一个用跟服务器通信的 bufferevent 事件对象。
3. 使用 bufferevent_socket_connect() 连接 服务器。
4. 使用 bufferevent_setcb() 给 bufferevent 对象的 read、write、event 设置回调。
5. 设置 bufferevent 对象的读写缓冲区 enable / disable。
6. 接受、发送数据 bufferevent_read() / bufferevent_write()。
7. 启动循环监听 event_base_dispatch()。
8. 释放资源。

代码实现

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
```

```
#include <string.h>
#include <event2/event.h>
#include <event2/listener.h>
#include <event2/bufferevent.h>

// bev对象的读写回调
void read_cb(struct bufferevent *bev, void *arg)
{
    char buf[1024];

    // 从客户端中读
    bufferevent_read(bev, buf, sizeof(buf));
    printf("client say:%s\n", buf);

    char *p = "我是服务器, 已经成功收到你发送的数据";
    // 写数据给客户端
    bufferevent_write(bev, p, strlen(p) + 1);

    sleep(1);
}

void write_cb(struct bufferevent *bev, void *arg)
{
    printf("I'm 服务器,成功写数据给客户端了,写缓冲区回调函数被调用...\n");
}

// listener监听器的回调函数
void listener_cb(struct evconnlistener *listener,
                 evutil_socket_t fd,
                 struct sockaddr *addr,
                 int len, void *ptr)
{
    printf("connect new client");

    // 接收监听器传进来的base
    struct event_base *base = (struct event_base *)ptr;

    // 创建bev对象,用于监听客户端的读写事件
    struct bufferevent *bev;
    bev = bufferevent_socket_new(base, fd, BEV_OPT_CLOSE_ON_FREE);

    // 设置bev对象的读写回调
    bufferevent_setcb(bev, read_cb, write_cb, NULL, NULL);

    // 设置读的权限
    bufferevent_enable(bev, EV_READ);
}

int main()
{
    // 地址结构体
    struct sockaddr_in serve_addr;
    serve_addr.sin_family = AF_INET;
    serve_addr.sin_port = htons(9527);
```

```

    serve_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    // 1. 创建底座
    struct event_base *base = event_base_new();
    // 2. 创建listener监听对象,(监听是否有客户端连接)
    struct evconnlistener *listener;
    listener = evconnlistener_new_bind(base, listener_cb, base,
                                       LEV_OPT_CLOSE_ON_FREE | LEV_OPT_REUSEABLE,
                                       -1, (struct sockaddr *)&serve_addr,
                                       sizeof(serve_addr));
    // 3. 启动循环监听
    event_base_dispatch(base);

    // 4. 释放
    event_base_free(base);           // 底座
    evconnlistener_free(listener); // 监听器

    return 0;
}

```

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>
#include <event2/event.h>
#include <event2/listener.h>
#include <event2/bufferevent.h>
#include <arpa/inet.h>

// 读写回调
void read_cb(struct bufferevent *bev, void *arg)
{
    char buf[1024] = {0};
    bufferevent_read(bev, buf, sizeof(buf));

    printf("服务器 say:%s\n", buf);
    bufferevent_write(bev, buf, strlen(buf) + 1);
    sleep(1);
}

void write_cb(struct bufferevent *bev, void *arg)
{
    printf("-----我是客户端的写回调函数,没卵用\n");
}

void event_cb(struct bufferevent *bev, short events, void *arg)
{
    if (events & BEV_EVENT_EOF)
    {

```

```
        printf("connection closed\n");
    }
    else if (events & BEV_EVENT_ERROR)
    {
        printf("some other error\n");
    }
    else if (events & BEV_EVENT_CONNECTED)
    {
        printf("已经连接服务器...\n");
        return;
    }
    // 释放资源
    bufferevent_free(bev);
}

int main()
{
    // 1. 创建base底座
    struct event_base *base = event_base_new();

    // 描述符
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    // 地址结构
    struct sockaddr_in client_addr;
    memset(&client_addr, 0, sizeof(client_addr));
    client_addr.sin_family = AF_INET;
    client_addr.sin_port = htons(9527);
    inet_pton(AF_INET, "127.0.0.1", &client_addr.sin_addr.s_addr);

    // 2. 创建bev对象
    struct bufferevent *bev = NULL;
    bev = bufferevent_socket_new(base, fd, BEV_OPT_CLOSE_ON_FREE);

    // 3. 连接服务器
    bufferevent_socket_connect(bev, (struct sockaddr *)&client_addr,
    sizeof(client_addr));

    // 4. 设置回调
    bufferevent_setcb(bev, read_cb, write_cb, event_cb, NULL);

    // 设置读回调生效
    bufferevent_enable(bev, EV_READ);

    // 5. 启动循环监听
    event_base_dispatch(base);

    // 6. 销毁
    event_base_free(base);
    bufferevent_free(bev);

    return 0;
}
```

web大作业